深度研究报告

video-note 测试体系现状审计

runtime repo 与 thin wrapper 的覆盖盘点、缺口分析与增量测试路线图

Codex

2026-03-26

核心问题：我们现在到底有哪些测试，它们实际保护了什么风险，以及下一批最值得增加哪些测试?

研究范围：以 `/home/fanghaotian/Desktop/src/video-note-pipeline` 为主，结合 `/home/fanghaotian/Desktop/src/agent-basic-skill/skills/video-note-render-pdf` 的 wrapper 边界。证据窗口截止 2026-03-26。

目标读者：video-note runtime / wrapper 的维护者，以及准备继续扩展 subtitle-first 与 live E2E 回归的人。

证据窗口：本地源代码、文档、pytest 实测日志与最新 live E2E 产物。

目录

1 执行摘要 **3**

2 研究范围与方法 **3**

3 当前测试版图 **5**

3.1 四层测试栈 5

3.2 Runtime repo：43 个 pytest 用例 5

3.3 Wrapper repo：17 个 pytest 用例 6

3.4 Live E2E：当前已经覆盖的真实 case 6

4 这些测试目前真正保护了什么 **7**

4.1 命令编排和工件契约已经比较稳 7

4.2 mode routing 已经有“单测 + live case”的双重保护 7

4.3 wrapper 侧保护的是“接线正确”，不是“语义正确” 7

5 当前明显未被充分保护的风险 **8**

5.1 未被 direct test 覆盖的关键模块 8

5.2 真实 subtitle-first 只在 mock 层被证明，没有在 live 层被证明 8

5.3 当前没有 CI 痕迹，意味着通过状态不是持续保证 8

6 应该增加什么测试 **9**

6.1 优先级排序 9

6.2 建议 1：把 subtitle-first 真正跑成 live 回归 9

6.3 建议 2：为 `transcripts.py` 建立 direct parser 回归 10

6.4 建议 3：为 adapter 层建立 contract tests 10

6.5 建议 4：把 `probes/subtitles.py` 单独拉出来测 10

6.6 建议 5：补齐 CLI 语义测试 11

6.7 建议 6：补一个 wrapper/runtime contract sync test 11

6.8 建议 7：把 build 的失败路径也锁住 11

6.9 建议 8：没有 CI，就把“该跑什么”写死到 CI 12

7 结论与后续问题 **12**

7.1 已经有强证据支持的结论 12

7.2 比较可能成立，但还没有被完全证明的结论 12

7.3 仍然未知，值得下一轮验证的问题 12

A 来源清单 **13**

B 附件说明 **14**

1 执行摘要

结论 1：现有测试资产并不薄弱，问题在于分布不均

截至本次审计，`/home/fanghaotian/Desktop/src/video-note-pipeline` 的运行时测试实测为 **43/43 通过**，`/home/fanghaotian/Desktop/src/agent-basic-skill` 中与 `video-note-render-pdf` 直接相关的 wrapper 测试为 **17/17 通过**，最新 live E2E canary 为 **3/3 case 通过**。因此当前状态不是“几乎没测试”，而是已经形成了 **单元/契约层、命令编排层、wrapper 解析层、live E2E 层**四层结构。

结论 2：最强保护层是命令编排与工件契约，最弱保护层是平台真实边界

当前最扎实的部分，是 `prepare` / `probe` / `transcript` / `overview` / `build` 这条命令编排链条，以及 case artifact contract 的稳定性。当前最弱的部分，是 **adapter 真正面对平台 API 漂移的解析层、transcript parser 本身、CLI 错误语义**，以及 **真实 subtitle 路径的 live E2E**。这意味着系统内部 glue code 已经相对稳，但最靠近外部依赖和最终用户入口的地方仍然偏薄。

结论 3：下一批最值得加的不是“更多同类 mock 测试”，而是补齐真实边界

按照本报告的优先级模型，最值得新增的测试依次是：

- 真实平台字幕路径的 live canary：至少一条公开字幕 case 和一条 cookies 字幕 case
- `transcripts.py` 的 direct parser tests
- `adapters/*.py` 与 `probes/subtitles.py` 的 direct contract tests
- `cli.py` 的命令行语义测试
- wrapper 与 runtime 的 contract sync tests

重要限定

本次审计是在 `/home/fanghaotian/Desktop/src/video-note-pipeline` 的 **dirty worktree** 上完成的；因此结论反映的是当前本地工作副本，而不是某个已打 tag 的发布版本。相关状态见附件 `runtime-git-status.txt`。

2 研究范围与方法

最小合理假设

用户没有单独限定“只看 runtime”或“只看 wrapper”，因此本报告采取最小合理范围：以 `video-note-pipeline` 为主分析对象，把 `video-note-render-pdf` wrapper 视为跨仓边界，并明确区分“runtime 内部测试”和“wrapper 边界测试”。

本次审计实际使用了三类证据：

- **源码与测试文件**：例如 `/home/fanghaotian/Desktop/src/video-note-pipeline/tests/test`

_commands.py、/home/fanghaotian/Desktop/src/video-note-pipeline/tests/test_cookies.py、/home/fanghaotian/Desktop/src/agent-basic-skill/tests/test_wrapper_resolution.py

- **文档与契约**：例如 /home/fanghaotian/Desktop/src/video-note-pipeline/README.zh-CN.md、/home/fanghaotian/Desktop/src/agent-basic-skill/skills/video-note-render-pdf/references/adapter-contract.md
- **实测结果**：本次审计保存的 runtime-pytest.log、wrapper-pytest.log，以及已有的 /home/fanghaotian/Desktop/src/video-note-pipeline/reports/live-e2e/runs/bilibili-core-base-model-rerun/summary.json

```
uv run pytest /home/fanghaotian/Desktop/src/video-note-pipeline/tests
uvx pytest tests/test_wrapper_resolution.py tests/test_install_skill.py tests/
    test_skill_repo.py
```

Listing 1: 本次审计实际执行的关键验证命令

为了给“应该先加哪些测试”排序，我使用了一个显式的优先级函数：

$$P = 2I + B + D - C$$

- P：建议测试的优先级分数，越高越该先做
- I：如果该区域出问题，对主链路和归档质量的影响，取值 1 到 5
- B：是否跨越 repo、平台或外部工具边界，取值 1 到 3
- D：外部漂移风险，例如 yt-dlp、平台字幕接口、浏览器 cookies 行为，取值 1 到 3
- C：新增该测试的工程成本，取值 1 到 3

这个公式不是现成标准，而是本次审计的 **方法学工具**。它的目的，是避免建议列表退化成“把所有未测试模块都补一遍”的平均主义。

3 当前测试版图

3.1 四层测试栈

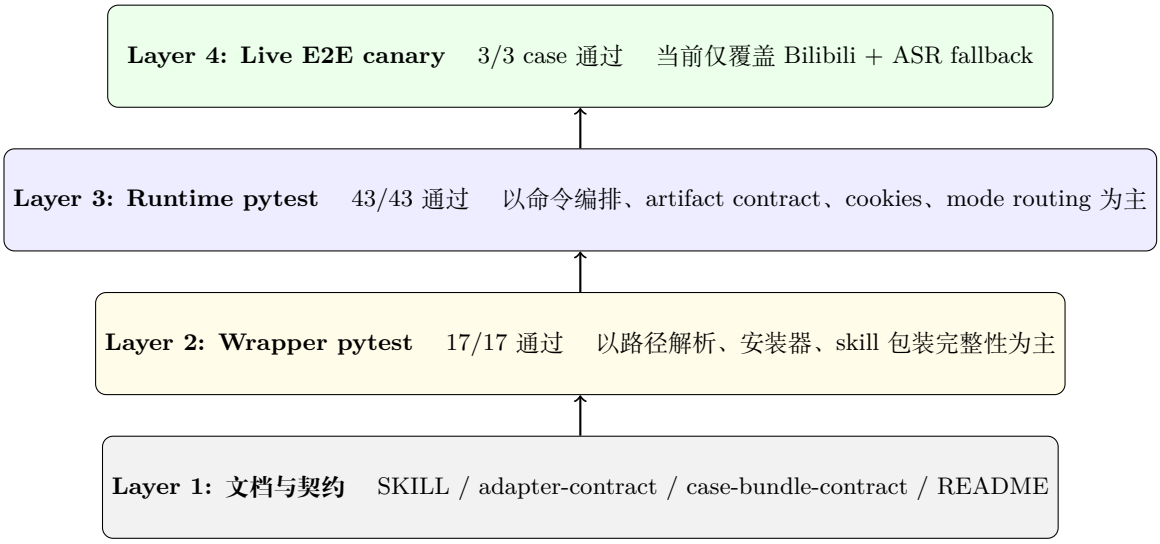

图 1: 当前测试栈示意图。

来源：*runtime-pytest.log*、*wrapper-pytest.log*、*/home/fanghaotian/Desktop/src/video-note-pipeline/reports/live-e2e/runs/bilibili-core-base-model-rerun/summary.json*，作者自绘。

3.2 Runtime repo：43 个 pytest 用例

测试文件	数量	主要保护的风险	当前仍明显缺的部分
`test_paths.py`	6	URL 平台识别、video id 抽取、workspace/config root 解析	host 变体与奇怪 URL 形式还不够多
`test_contracts.py`	3	artifact 索引、inventory 结果、case contract 稳定性	没有跨 repo 的 schema sync 保护
`test_cookies.py`	9	keyring 选择、cookies 导出、平台过滤、去重、状态提示	真实浏览器 decrypt 和真实登录态仍未进入自动化
`test_transcriber_probe.py`	1	transcriber candidate 排序与 skip 逻辑	真正的 CUDA / MLX sample 行为仍主要靠本机人工验证
`test_frames.py`	4	montage 输出与 `recommended_mode` 路由启发式	真实视频分布仍只覆盖有限 archetype
`test_build.py`	2	build happy path、外部 `note.tex` 支持	latexmk 失败、preview 分支、缺失 PDF 分支没单测
`test_commands.py`	11	命令编排、stage 更新、subtitle/cookies/ASR fallback、manifest 更新	仍以 fake adapter 为主，外部平台解析被 mock 掉了
`test_live_e2e.py`	4	manifest 解析、case evaluation、smoke note 生成	只测 harness 自身，不测真实网络视频

表 1: runtime repo 的当前 pytest 结构。

计数来源：*runtime-test-counts.csv* 与 *runtime-pytest.log*。

3.3 Wrapper repo：17 个 pytest 用例

测试文件	数量	主要保护的风险	当前仍明显缺的部分
`test_wrapper_resolution.py`	7	runtime repo 与 workspace 路径解析优先级	只验证路径，不验证运行后 contract 是否匹配
`test_install_skill.py`	5	安装器能识别外部 runtime repo，并正确复制 wrapper skill	不验证安装后的真实 runbook 与 runtime 兼容性
`test_skill_repo.py`	5	skill 包装目录包含预期文件、wrapper 结构完整	不验证文档字段与 runtime 实现是否语义同步

表 2: wrapper 侧与 `video-note-render-pdf` 直接相关的当前 pytest 结构。

计数来源：*wrapper-test-counts.csv* 与 *wrapper-pytest.log*。

3.4 Live E2E：当前已经覆盖的真实 case

当前 live E2E manifest 位于 `/home/fanghaotian/Desktop/src/video-note-pipeline/live_e2e/bilibili-core.json`，最新一次完整结果位于 `/home/fanghaotian/Desktop/src/video-note-pipeline/reports/live-e2e/runs/bilibili-core-base-model-rerun/summary.json`。

Case	预期模式与覆盖价值	当前观测到的 transcript / subtitle 路径	结果
`BV1GZw9zcEdJ`	`visual-light`，代表轻量口播/低视觉密度	`subtitle_source = none`，`transcript_source = asr:faster-whisper:cpu`	PASS
`BV1cptBzFE5Q`	`static-outline`，代表单画布大纲课件	`subtitle_source = none`，`transcript_source = asr:faster-whisper:cpu`	PASS
`BV1G5AAzHEH3`	`board-heavy`，代表高视觉密度屏录/白板	`subtitle_source = none`，`transcript_source = asr:faster-whisper:cpu`	PASS

表 3: 当前 live E2E canary 已覆盖的三类 archetype。

来源：*/home/fanghaotian/Desktop/src/video-note-pipeline/reports/live-e2e/runs/bilibili-core-base-model-rerun/summary.md*。

这里的关键限制

这套 live E2E 虽然很有价值，但它目前 **只证明了 ASR fallback 主链路**。它还没有自动证明：

- 公开平台字幕路径在真实视频上持续可用
- cookies 字幕路径在真实登录态下持续可用
- YouTube 与 Bilibili 的平台差异在 live 环境中同时稳定

4 这些测试目前真正保护了什么

4.1 命令编排和工件契约已经比较稳

证据链

`/home/fanghaotian/Desktop/src/video-note-pipeline/tests/test_commands.py` 覆盖了 `prepare` / `probe` / `transcript` / `overview` / `build` / `cookies-export` 六类命令的主要 happy path 与关键 fallback；`/home/fanghaotian/Desktop/src/video-note-pipeline/tests/test_contracts.py` 则确保 artifact 索引和 inventory 不会无声漂移。两者结合，意味着 case 生命周期的“骨架”已经被有效固定住。

如果只从归档风险看，当前最不容易被无声破坏的部分，是：

- case 目录结构与 artifact 命名
- stage 状态推进
- subtitle-first 在命令层的编排顺序
- build 产物落位
- `recommended_mode` 写回 `preflight.json` 与 manifest

4.2 mode routing 已经有“单测 + live case”的双重保护

证据链

`/home/fanghaotian/Desktop/src/video-note-pipeline/tests/test_frames.py` 提供启发式单测；`/home/fanghaotian/Desktop/src/video-note-pipeline/reports/live-e2e/runs/bilibili-core-base-model-rerun/summary.json` 证明真实视频下三类 archetype 都能得到当前预期模式。

这意味着 `talking-head` / `visual-light` / `static-outline` / `board-heavy` 这套路由体系，已经不再只是 README 里的设计词汇，而是有运行时实证支撑的行为层输出。

4.3 wrapper 侧保护的是“接线正确”，不是“语义正确”

证据链

`/home/fanghaotian/Desktop/src/agent-basic-skill/tests/test_wrapper_resolution.py`、`/home/fanghaotian/Desktop/src/agent-basic-skill/tests/test_install_skill.py`、`/home/fanghaotian/Desktop/src/agent-basic-skill/tests/test_skill_repo.py` 证明 wrapper 能找到 runtime repo、能按安装器要求落盘、也包含所需参考文件。

但这些测试目前并不证明以下语义层断言：

- wrapper 文档里声明的必要 artifact 是否始终和 runtime 输出一致

- wrapper 的 subtitle-first 文字约束是否与 runtime 的真实 fallback 行为持续同步
- wrapper runbook 中的命令示例是否始终和 runtime CLI 参数保持一致

5 当前明显未被充分保护的风险

5.1 未被 direct test 覆盖的关键模块

根据附件 `source-module-test-map.txt`，以下模块目前没有独立的 direct test file：

模块	为什么重要	现在的问题
`adapters/base.py`、`adapters/youtube.py`、`adapters/bilibili.py`	它们直接面对 yt-dlp 与 YouTubeTranscriptApi，是最容易被外部平台漂移打穿的边界	当前大部分命令测试都 fake 掉 adapter，因此平台解析层本身缺少稳定保护
`probes/subtitles.py`	这里编码了匿名字幕、cookies 字幕、失败原因拼接、selected payload 落盘等逻辑	虽然命令层间接覆盖到一部分，但没有更细粒度的 direct assertions
`transcripts.py`	统一 transcript 三件套的 parser 与 writer 都在这里，是事实层核心格式化模块	目前没有 `test_transcripts.py` 一类的 direct parser tests
`cli.py`	这是最终用户真正会调用的命令入口	当前没有 direct CLI parser / exit code / JSON output tests
`utils/io.py`、`utils/process.py`	是若干命令的底层依赖	风险较低，但一旦出问题会影响多个命令

表 4: 当前没有 direct test file 的主要模块。

来源：`source-module-test-map.txt`。

5.2 真实 subtitle-first 只在 mock 层被证明，没有在 live 层被证明

这是当前最重要的测试缺口。原因不在于文档没写，而在于：

- wrapper 文档 `/home/fanghaotian/Desktop/src/agent-basic-skill/skills/video-note-render-pdf/SKILL.md` 明确要求先匿名字幕、再 cookies、最后 ASR
- runtime 命令层的测试也已经验证了这种编排
- 但 live E2E manifest 里的 3 个 case 全部走的是 `subtitle_source = none` 与 `transcript_source = asr:faster-whisper:cpu`

因此，当前最强的结论是："ASR fallback 主链路被 live 证明了"；而不是"subtitle-first 的三层路径都已经被 live 证明了"。

5.3 当前没有 CI 痕迹，意味着通过状态不是持续保证

本次审计没有在两个仓库里发现 `.github/workflows/` 下的工作流文件，也没有发现 `Makefile`、`justfile`、`tox.ini` 或 `noxfile.py` 这类标准化测试入口。换句话说，当前测试是"可运行的"，但还

不算“持续执行的”。这会直接降低新增测试的实际价值，因为没有自动门禁时，再好的测试也容易只在本机偶尔跑一次。

6 应该增加什么测试

6.1 优先级排序

优先级	建议测试	分数 P	推荐落点	为什么现在加
P0	真实公开字幕 canary + 真实 cookies canary	11	`live_e2e/` manifest、`scripts/run_live_e2e.py`	当前 live 只覆盖 ASR fallback，最重要的规范路径还没有 real-world 证明
P0	`transcripts.py` direct parser tests	10	新增 `tests/test_transcripts.py`	transcript 是事实层核心，但当前完全没有 direct tests
P0	adapter contract tests	10	新增 `tests/test_adapters.py`	这里最靠近 yt-dlp / YouTubeTranscriptApi，是平台漂移首个冲击面
P1	`probes/subtitles.py` direct tests	9	新增 `tests/test_subtitle_probe.py`	命令层已覆盖编排，但 probe 模块本体仍缺细粒度保护
P1	CLI 语义测试	8	新增 `tests/test_cli.py`	当前最终用户入口没有 direct tests，参数解析和 exit code 漂移无门禁
P1	wrapper/runtime contract sync tests	8	`agent-basic-skill/tests/` 新增 contract sync file	现有 wrapper 测试只保结构，不保语义同步
P1	build failure / preview 分支测试	7	扩展 `tests/test_build.py`	当前 build 主要保护 happy path，失败路径仍偏薄
P2	resume / rebuild / broken artifact recovery tests	7	新增 `tests/test_commands_recovery.py` 或扩展现有命令测试	这类问题在真实工作流里常见，但当前覆盖不足

表 5: 基于 $P = 2I + B + D - C$ 的新增测试排序。

6.2 建议 1：把 subtitle-first 真正跑成 live 回归

这不是再加一个 mock 测试，而是把当前 live 套件从“只验证 ASR fallback”升级成“验证整个 subtitle-first 梯度”。具体建议如下：

- 增加一个 **公开视频字幕** case：例如稳定可访问的 YouTube 公共字幕视频，期望 `selected_source = official-anonymous`
- 增加一个 **cookies 字幕** case：由用户本机已登录浏览器 profile 提供登录态，期望 `selected_source = official-cookies`
- 继续保留当前 3 个 ASR-only Bilibili archetype，作为 mode routing 与 build 主链路 smoke set

为什么这是第一优先级

因为系统设计从 README 到 wrapper runbook 都强调 subtitle-first。如果 live 层永远只跑 ASR fallback，那么当前最重要的设计约束实际上仍然停留在 mock 证明阶段。

6.3 建议 2：为 `transcripts.py` 建立 direct parser 回归

建议新增 `tests/test_transcripts.py`，至少覆盖以下场景：

- `parse_srt_text`：标准 SRT、多行字幕、空白块、逆序 end time
- `parse_vtt_text`：带 `WEBVTT` 头、块间空行、多行正文
- `parse_json3_payload`：YouTube JSON3 events 中空文本、零时长、多个 seg 拼接
- `write_transcript_outputs` round-trip：写出的 `transcript.json` / `.srt` / `.txt` 是否与 payload 一致

这是高优先级，不是因为代码多，而是因为它处在“所有上游来源最终统一汇聚的单点”。一旦这里出错，后续写作、选图、时间窗对齐都会被污染。

6.4 建议 3：为 adapter 层建立 contract tests

建议新增 `tests/test_adapters.py`，重点不是测网络，而是测 **contract**：

- `choose_preferred_track` 是否始终 manual 优先于 auto、中文优先于英文
- `normalize_metadata` 的缺省字段与 fallback 行为
- `summarize_best_format` 的格式选择逻辑
- `_resolve_subtitle_sidecar` 是否正确定位 `.json3` / `.srt` / `.vtt`
- YouTube adapter 对 `YouTubeTranscriptApi` 返回对象的转换
- Bilibili adapter 对 yt-dlp sidecar 的下载与解析约定

当前命令层测试把 adapter 大量 fake 掉，这对编排层是对的，但也让平台边界层成了一个明显空洞。

6.5 建议 4：把 `probes/subtitles.py` 单独拉出来测

建议新增 `tests/test_subtitle_probe.py`。至少应覆盖：

- 匿名成功时不再尝试 cookies
- 匿名返回 `selected_track` 但 payload 不可用时，仍继续尝试 cookies
- cookies 也失败时，`selected_track.source = none` 且 reason 合并清晰

- selected payload 会被正确写入 `selected_subtitle.json`

这类测试看起来和 `test_commands.py` 有重叠，但价值在于：一旦 fallback 规则变复杂，probe 模块的 direct tests 会比命令级别 tests 更容易定位问题。

6.6 建议 5：补齐 CLI 语义测试

建议新增 `tests/test_cli.py`，至少包括：

- 子命令 parser 是否接受预期参数
- `--json` 输出是否是稳定 JSON
- `CommandError` / `FileNotFoundError` 是否返回预期退出码
- `build --note-tex`、`overview --sample-count` 等选项是否正确转发

当前系统的“真正用户入口”是 CLI，而不是内部 Python API。如果 CLI 不受保护，用户感知到的稳定性会低于内部代码质量。

6.7 建议 6：补一个 wrapper/runtime contract sync test

建议在 `/home/fanghaotian/Desktop/src/agent-basic-skill/tests/` 里新增专门针对 `video-note-render-pdf` 的 contract sync 测试，做的事情不是运行视频，而是比较：

- wrapper 文档中声明的必要 artifact
- runtime `contracts.py` 里定义的 artifact 文件与目录
- runtime 命令真实写回的关键字段，例如 `preflight.json.recommended_mode`

原因很简单：当前 wrapper 测试已经证明“这个 skill 能装、能解析路径、文件都在”，但还没有证明“它说的话和 runtime 真的一致”。

6.8 建议 7：把 build 的失败路径也锁住

当前 `tests/test_build.py` 已经覆盖 happy path 与外部 `note.tex`。建议继续增加：

- latexmk 返回非零时，`build_summary.json` 是否如实标记失败
- `note.tex` 缺失时，错误信息是否足够明确
- 系统存在 `pdftoppm` 时，preview 图片是否正确落盘
- 外部 `note.tex` 存在但没有生成 PDF 时，是否给出可定位的失败信号

6.9 建议 8：没有 CI，就把“该跑什么”写死到 CI

如果只谈新增测试而不谈执行入口，收益会被打折。建议的最小 CI 方案是：

- runtime repo：默认 PR 上跑 `uv run pytest`
- wrapper repo：默认 PR 上跑 `uvx pytest tests/test_wrapper_resolution.py tests/test_install_skill.p` `tests/test_skill_repo.py`
- live E2E：先不做强制门禁，但保留一个手动触发 workflow，至少跑公开字幕 canary 与当前 3 个 ASR smoke case

7 结论与后续问题

7.1 已经有强证据支持的结论

- 当前不是“缺少测试的原型仓库”，而是已有较清晰分层的测试体系
- `video-note-pipeline` 当前工作副本的 runtime pytest 实测为 43/43 通过
- `agent-basic-skill` 中与 `video-note-render-pdf` 直接相关的 wrapper pytest 实测为 17/17 通过
- live E2E 已经覆盖 `visual-light`、`static-outline`、`board-heavy` 三类 archetype，并且 3/3 通过

7.2 比较可能成立，但还没有被完全证明的结论

- subtitle-first 的匿名字幕路径与 cookies 路径在真实 live 环境中长期稳定
- adapter 面对 yt-dlp / YouTubeTranscriptApi 行为漂移时仍能保持 contract 稳定
- wrapper 文档与 runtime contract 在未来演进中不会重新漂移

7.3 仍然未知，值得下一轮验证的问题

- 真实登录态 cookies case 是否能稳定进入自动化，而不把用户本地浏览器环境弄得太脆
- transcript parser 在真实混杂字幕格式下的边角行为是否已经足够稳
- 在没有 CI 的情况下，当前通过状态能否被持续维持，而不是依赖人工记忆

如果只允许我给一个最简路线图，我会建议按下面顺序推进：

1. 先把公开字幕 canary 和 cookies canary 加进 live E2E
2. 再补 `tests/test_transcripts.py` 与 `tests/test_adapters.py`
3. 接着补 `tests/test_cli.py` 与 wrapper/runtime contract sync test
4. 最后把 runtime pytest 与 wrapper pytest 接进 CI

A 来源清单

ID	模态	标题 / 标签	可信度	链接或路径
S1	Code	runtime 依赖与 pytest 配置	primary	/home/fanghaotian/Desktop/src/video-note-pipeline/pyproject.toml
S2	Doc	runtime README 与 subtitle-first 说明	primary	/home/fanghaotian/Desktop/src/video-note-pipeline/README.zh-CN.md
S3	Code	runtime 命令编排测试	primary	/home/fanghaotian/Desktop/src/video-note-pipeline/tests/test_commands.py
S4	Code	cookies 测试	primary	/home/fanghaotian/Desktop/src/video-note-pipeline/tests/test_cookies.py
S5	Code	frames / mode routing 测试	primary	/home/fanghaotian/Desktop/src/video-note-pipeline/tests/test_frames.py
S6	Code	build 测试	primary	/home/fanghaotian/Desktop/src/video-note-pipeline/tests/test_build.py
S7	Code	transcriber probe 测试	primary	/home/fanghaotian/Desktop/src/video-note-pipeline/tests/test_transcriber_probe.py
S8	Code	live E2E harness 测试	primary	/home/fanghaotian/Desktop/src/video-note-pipeline/tests/test_live_e2e.py
S9	Data	最新 live E2E 汇总	primary	/home/fanghaotian/Desktop/src/video-note-pipeline/reports/live-e2e/runs/bilibili-core-base-model-rerun/summary.json
S10	Code	cookies helper 当前实现	primary	/home/fanghaotian/Desktop/src/video-note-pipeline/src/video_note_pipeline/cookies.py
S11	Code	wrapper 路径解析测试	primary	/home/fanghaotian/Desktop/src/agent-basic-skill/tests/test_wrapper_resolution.py
S12	Code	wrapper 安装测试	primary	/home/fanghaotian/Desktop/src/agent-basic-skill/tests/test_install_skill.py
S13	Code	wrapper 包装结构测试	primary	/home/fanghaotian/Desktop/src/agent-basic-skill/tests/test_skill_repo.py
S14	Doc	wrapper 主说明	primary	/home/fanghaotian/Desktop/src/agent-basic-skill/skills/video-note-render-pdf/SKILL.md
S15	Doc	wrapper adapter contract	primary	/home/fanghaotian/Desktop/src/agent-basic-skill/skills/video-note-render-pdf/references/adapter-contract.md
S16	Data	runtime pytest 审计日志	primary	/home/fanghaotian/Desktop/src/agent-basic-skill/reports/2026-03-26-video-note-testing-audit/runtime-pytest.log
S17	Data	wrapper pytest 审计日志	primary	/home/fanghaotian/Desktop/src/agent-basic-skill/reports/2026-03-26-video-note-testing-audit/wrapper-pytest.log

ID	模态	标题 / 标签	可信度	链接或路径
S18	Data	源码模块到测试文件映射	secondary	/home/fanghaotian/Desktop/src/agent-basic-skill/reports/2026-03-26-video-note-testing-audit/source-module-test-map.txt
S19	Data	runtime dirty worktree 状态	secondary	/home/fanghaotian/Desktop/src/agent-basic-skill/reports/2026-03-26-video-note-testing-audit/runtime-git-status.txt

B 附件说明

本报告目录下还保留了以下审计附件，便于后续追溯：

- runtime-pytest.log
- wrapper-pytest.log
- runtime-test-counts.csv
- wrapper-test-counts.csv
- source-module-test-map.txt
- runtime-git-status.txt
- wrapper-git-status.txt